

```
#Train
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

import joblib

# Load dataset
df = pd.read_csv("email.csv")

# Features and target
X = df["Message"]
y = df["Category"]

# Encode labels (spam/ham → 0/1)
le = LabelEncoder()
y = le.fit_transform(y)

# Convert text → numerical
cv = CountVectorizer()
X = cv.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Models
models = {
    "KNN": KNeighborsClassifier(),
    "SVM": SVC(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier()
}

best_model = None
best_accuracy = 0
best_name = ""
```

```

print("Model Accuracy:\n")

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    print(f"{name}: {acc*100:.2f}%")

    if acc > best_accuracy:
        best_accuracy = acc
        best_model = model
        best_name = name

print(f"\nBest Model: {best_name} ({best_accuracy*100:.2f}%)")

# Save model + vectorizer + encoder
joblib.dump(best_model, "spam_model.pkl")
joblib.dump(cv, "vectorizer.pkl")
joblib.dump(le, "label_encoder.pkl")

print("Model saved")

```

```

Model Accuracy:

KNN: 92.56%
SVM: 98.12%
Decision Tree: 97.04%
Random Forest: 98.12%

Best Model: SVM (98.12%)
Model saved

```

Spam Detection App

hello

Predict

Prediction: Not Spam